

Proposal

Proposal for Orbital 26

Team Name:

[Please come up with a creative name for your team. Your team name should contain at least one English letter.]

MahjongSensei

Proposed Level of Achievement:

[Project Gemini or Apollo 11 or Artemis. Please read the descriptions of the levels of achievement on the Orbital website: <https://soc-n.us/orbital-loa> and choose only one.]

Apollo 11

Motivation

Our team has had a passion for playing Mahjong, with Dylan having started MahjongTogether, a community based Mahjong session with the elderly in 2022. We want to explore deeper into using our skills to help Mahjong be more accessible to more people, and share our love for the game with everyone.

Aim

We hope to help more people start playing Mahjong by making a beginner friendly and interactive tool available to more people, and track their playing styles, while also teaching them how to play optimally.

User Stories

[Please describe what the users would be able to do with your system.]

1. As a beginner to Mahjong who wants to learn how to play, I want to be able to learn the rules and fundamentals of the game from the app.
2. As an intermediate at Mahjong who wants to hone my skills, I want to be able to train my decision-making from the app

Features

[Please list down the key features of your system with a rough timeline for completion.]

1. Introduction to Mahjong rules and points system
 - a. teaches users the basics of winning a mahjong hand
 - b. teaches users alternative ways to win a hand
 - c. teaches users points system
2. Mahjong Hand Helper, user inputs a hand and the app will suggest a recommended move

3. Interactive Trainer, app will generate a hand and test users on best move to make, real time feedback using valuation algorithm
4. Soloplay feature, for user to play against the computer

Timeline

[Please provide a rough timeline for completion in terms of Milestones.]

1. Milestone 1 (1 June) - Technical proof of concept (minimal working system with both the frontend and the backend integrated for a very simple feature)
 - a. Basic front end of app completed
 - b. Back end: MAHJONG tiles OOP completed
 - c. Feature 1 completed plus its basic front end designs.
2. Milestone 2 (29 June)- Prototype (i.e., a working system with the core features)
 - a. Feature 2 completed plus its basic front end designs
 - b. Feature 3 without AI implementation, workout generation of mahjong hands
3. Milestone 3 (27 July) - Extended system (i.e., a working system with both the core + extension features)
 - a. Integration of AI into Feature 3
 - b. Feature 4 (making user dashboard with AI integrated to track habits/playstyle)

Tech Stack

[Please list down the technologies that you are planning to use.]

1. React
2. FastAPI
3. Python

Qualifications

[Please elaborate on your technical skills and project experience of your team. This is required only for teams targeting Apollo 11 or Artemis.]

Both team members:

- completed CS1010A & CS2030
- made personal projects ranging from habit trackers to finance options calculators.

Software Engineering

We will develop a layered system to ensure the different functionalities are decoupled to ensure maintainability and scalability of the system.

- React will handle user interaction
- FastAPI handles request handling and communication
- Python handles all game logic
- Use of AI to handle Monte Carlo simulations and user advice

We will implement multiple levels of testing

- Unit testing to test individual components
 - Hand validation logic
 - Tile combination generation
 - Suggestion algorithms
- Integration testing
 - API endpoints and logic engine
 - Frontend and backend interaction
- Edge case testing
 - Invalid hands
 - Rare tile combinations
 - Boundary cases in probability calculations

We will make use of React testing library and Pytest for frontend and backend respectively

Research

Advisor points:

- Add more features (maybe add a solo play against AI)
- README (10 pages for apollo)
- Verify what we mean by AI (maybe look into existing models / try to find existing Mahjong API)
 - Look into simpler algorithm into assigning values to how good or bad something is (maybe can be worth multiple features)
- Maybe can for LLM can try chatbot
- Can add soloplay mode (count as a few features)
 - Can look into state machine
 - Can study mahjong games github repos / study source code
- Milestone 1
 - Basic user login
 - Basic feature (maybe halfway done tutorial)
 - Backend frontend
- Tech stack
 - Component libraries for frontend UIs
 - django/Flask for Python to setup links to frontend/DB
 - Lookup Model view controller architecture (simple way to view backend)
 - React use typescript/javascript
- Revise features (relook at AI stuff)
 - Integrate other people's work

Improved Feature 1

Feature 1 — Interactive Mahjong Learning & Rules Engine

A comprehensive interactive tutorial system where users can:

- learn Mahjong rules through guided visual walkthroughs,
- interact with dynamically rendered Mahjong hands and tile combinations,
- receive real-time validation and explanations of winning hands,
- explore scoring mechanics through a rule-based points calculation engine,
- and complete progressive learning modules with quizzes and scenario-based exercises.

Core technical implementations:

- Mahjong hand validation engine
- Tile state management system
- Dynamic scoring calculator
- Interactive tutorial rendering system
- Progress tracking and lesson persistence

2. Add user authentication:

The cleanest setup is:

Frontend

React:

- Login page
- Register page
- Session management

Backend

FastAPI:

- JWT authentication
- Password hashing (`bcrypt`)
- Protected API routes

Database

Store:

- users
- progress
- playstyle stats
- saved training sessions

User logs in

↓

Frontend sends credentials to FastAPI

↓

FastAPI verifies password



JWT token returned



Frontend stores token



Authenticated requests include token

3. Use Hybrid architecture

NOT:

- fully OOP-heavy Java-style design

YES:

- modular architecture
- some OOP where natural
- functional/service-based backend overall

Better Backend Style

Use:

Service-Based Architecture

Example:

```
/services
  scoring_service.py
  trainer_service.py
  ai_service.py
  auth_service.py

/models
  user.py
  tile.py
  hand.py

/routes
  auth_routes.py
  trainer_routes.py
```

This is MUCH more industry-like.

Recommended Architecture

Frontend

React + component-based UI

Backend

FastAPI modular API architecture

Logic

Mix of:

- lightweight OOP models
- functional/service-based processing

Database

PostgreSQL

ORM

SQLAlchemy

Auth

JWT

AI

Monte Carlo simulation + heuristic evaluation



This would make your project feel genuinely Apollo-tier.

Persistence means:

data still exists after the app closes.

Without persistence:

- everything resets,
- no saved users,
- no saved training history,
- no profiles.

6. Database Recommendation (Very Important)

You absolutely should add one.

Recommended Database

PostgreSQL

Best choice because:

- industry standard,
- works beautifully with FastAPI,
- relational data fits your app,
- Apollo evaluators like it.

Why NOT MongoDB

Your data is relational:

- users
- sessions
- hands
- strategies
- statistics



SQL fits better.

Suggested Tables (database)

Users

id
username
password_hash
created_at

TutorialsCompleted

user_id
lesson_id
completion_status

TrainingSessions

user_id
starting_hand
user_move

optimal_move
score
timestamp

PlaystyleAnalytics

user_id
aggression_score
efficiency_score
Risk_score

4. AI

AI-Assisted Decision Analysis System

The application will:

- evaluate player decisions using Mahjong heuristics and probability simulations,
- compare user choices against statistically optimal moves,
- identify recurring strategic patterns,
- and generate personalized improvement suggestions.

Potential implementations:

- Monte Carlo simulations
- Expected value calculations
- Rule-based heuristics
- Reinforcement-learning-inspired evaluation systems

This sounds MUCH stronger.

Applications to use

Layer	Technology
Frontend	React + Vite + TailwindCSS (communication with backend? Nextjs? Find Component libraries)
Backend API	FastAPI / flask or django (MVC architecture)
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	JWT + bcrypt
AI/Logic	Python
State Management	React Context
Deployment Frontend	Vercel
Deployment Backend	Railway / render
Database Hosting	Railway PostgreSQL
API Testing	Postman
Version Control	GitHub
UI Design	Figma

Liftoff

5 — User Profile & Progress Dashboard Since you already have PostgreSQL storing user data and move history, this is mostly a frontend task. Display stats like games played, win rate, most common discards, tutorial completion progress. Recharts makes the visualisation straightforward.

6 — Leaderboard A simple ranked table pulling win rates or trainer scores from the database. Minimal backend work since the data is already being stored — just a new API endpoint and a React table component.

7 — Hand History Log Let users review their past Hand Helper queries and Trainer attempts — what hand they had, what they chose, and what the correct move was. Pure database read, no new logic needed.

Medium effort but very achievable

8 — Difficulty Modes for the Trainer Add Easy / Medium / Hard tiers to Feature 3 by controlling how close to complete the generated hands are. Easy hands are one tile away from winning, Hard hands require more complex decision-making. Just a parameter on your hand generation logic.

9 — Hint System in Solo Play A "hint" button during Solo Play that calls the same valuation algorithm from Feature 2 and suggests the best move. Reuses existing code entirely — no new logic, just wiring the button to the existing Hand Helper endpoint.

10 — Tile of the Day / Daily Challenge Generate a fixed hand each day as a challenge for all users to attempt in Trainer mode. Seed the random hand generator with the current date so everyone gets the same hand. Simple to implement, adds a fun recurring engagement hook.

Project Log

1. **Prep Stage – 5 hours**(14 - 16 May)
 - a. 14 May (2 hours)
 - i. Dylan – Researched and settled on the optimal applications required for MahjongSensei app (considering our core features), for frontend, backend as well as the frameworks and programming language which we have decided to stick to (python).
 - ii. Anjie – Developed and designed the Poster & Liftoff Video, developed the scripts.
 - b. 15 May (1.5 hour)
 - i. Both – called with our advisor to discuss improvement pointers, gather tips and advice to move on with our project. Proceeded to change large portions of our proposal and project based on advice given.
 - c. 16 May (1.5 hour)
 - i. Both – Filmed liftoff video, finalised poster, submitted on skylab. Finalised core features and planned.
 - d. 18 May (2 hours)
 - i. Both - Prepared and structured code base, created local PostgreSQL database setup Django to link with React and local database
 - e. 19 May (3 hours)
 - i. Both - Preliminary setup for registration view and authentication routes for user login system. Created User serializer using Django user model, as well as created profile model and profile serializer.
 - f. 20 May (6 hours)
 - i. Both - Created Game Engine file with working Tile, Hand, Deck and Hand Generator classes. Completed working implementations for all different tiles including bamboo, characters, circles, honour and bonus tiles. Hand classes implemented with normal Tile List and Bonus Tile List, with discard and add_tile functions. Deck class implemented with build_deck class to generate a valid deck, and also implemented with shuffle and draw functions. Hand Generator class implemented with basic checks to replace any bonus (flower/animal) tiles with new ones so that hands generated have 13 playing tiles. Created and passed test cases for all of the above classes. Created global authentication states for login, logout and register functions. Configured axios to point at Django and auto-attach JWT token to all requests. Created and connected Login and Register forms to backend, and protected home page that only shows after login. Configured app.jsx to route with protected with protected routes using React router

2.

Milestone I Project Log

MahjongSensei
Chew An Jie | Dylan Wee

Stage	Date	Time Dedicated	Team Member(s)	Work Completed
Prep Stage	14 May	2 hours	Dylan	Researched and settled on the optimal applications required for MahjongSensei app (considering core features) for frontend, backend, frameworks, and programming language (Python).
		2 hours	Anjie	Developed and designed the Poster & Liftoff Video; developed the scripts.
	15 May	1.5 hours	Both	Called with advisor to discuss improvement pointers, gather tips, and advice. Changed large portions of the proposal and project based on the advice given.
	16 May	1.5 hours	Both	Filmed liftoff video, finalized poster, and submitted on SkyLab. Finalized core features and project planning.

Milestone 1 Work	18 May	2 hours	Both	Prepared and structured the codebase. Created local PostgreSQL database and set up Django to link with React and the local database.
	19 May	3 hours	Both	Preliminary setup for registration view and authentication routes for the user login system. Created User serializer using Django user model, as well as the profile model and profile serializer.
	20 May	6 hours	Both	Backend: Created Game Engine file with working Tile, Hand, Deck, and Hand Generator classes. Completed working implementations for all different tiles (bamboo, characters, circles, honour, and bonus). Hand classes implemented with normal Tile List and Bonus Tile List, featuring discard and add_tile functions. Deck class implemented with build_deck class to generate a valid deck, shuffle, and draw functions. Hand Generator class implemented with basic checks to replace bonus tiles so hands have 13 playing tiles. Created and passed test cases for all classes. Frontend & Auth: Created global authentication states for login, logout, and register functions. Configured Axios to point at Django and auto-attach JWT token to all requests. Created and connected Login and Register forms to the backend. Protected the home page to only show after login and configured app.jsx to route with protected routes using React router.

	22 May	3 hours	Both	API Integration & Frontend Hand Generation: Integrated the Python Mahjong engine (engine.py) with Django REST Framework. Created the authenticated random-hand API endpoint (views.py, urls.py). Developed Home.jsx on the frontend to fetch and display the generated 13-tile hand via Axios, styling the display inside index.css.
	25 May	4 hours	Both	Session Persistence & Route Protection: Upgraded AuthContext.jsx to read stored JWT tokens on page refresh, keeping users logged in seamlessly. Implemented a ProtectedRoute wrapper in App.jsx to block unauthenticated access to the dashboard and profile pages. Developed a unified AppLayout.jsx navigation shell component to prevent duplicated UI code across views.
	27 May	5 hours	Both	Profile Dashboard & System Testing: Created the backend profile serialization layer to pass user IDs, account creation timestamps, and game statistics. Built the frontend Profile.jsx view to render this data dynamically. Wrote and executed test suites for both frontend and backend modules (10 total test cases passing successfully). Verified production compilation via npm run build.
	29 May	4 hours	Both	Architecture Mapping & Git Cleanup: Drafted and mapped comprehensive system documentation in README.md, including frontend-backend system architecture, login/session JWT sequence workflows, database schemas, and an object-oriented class diagram for the Mahjong engine.

				Executed repository hygiene commands to strip local environment files (venv, .env, .DS_Store) from Git tracking history.
	30 - 31 May	4 hours	Both	Poster Design & Video Scripting
Total hours:		38 hours (per teammate)		

Starting stuff

gpThe big picture first

Before touching any code, here's what you're building and how the pieces fit together:

- **React** is what the user sees and clicks on in the browser
- **Django** is the brain behind the scenes — it receives requests from React, processes them, and sends back data
- **PostgreSQL** is the database — like a spreadsheet that permanently stores all your data

They're three separate things running on your computer at the same time during development. React and Django talk to each other over the internet (even locally), and Django talks to PostgreSQL to read and write data.

Phase 1 — Install everything you need

You only do this once.

1. Install Python Go to python.org, download the latest version (3.11 or 3.12), and install it. During installation on Windows, tick the box that says "**Add Python to PATH**" — this is important.

Check it worked by opening your terminal (Command Prompt on Windows, Terminal on Mac) and typing:

```
bash
python --version
```

It should print something like `Python 3.11.4`.

2. Install Node.js Go to nodejs.org and download the LTS version. This is needed to run React.

Check it worked:

```
bash
node --version
npm --version
```

3. Install PostgreSQL Go to postgresql.org/download, download and install it. During setup it will ask you to set a password — write this down, you'll need it later.

4. Install VS Code Go to code.visualstudio.com and install it. This is the editor you'll write your code in. Once installed, open it and install these two extensions (click the Extensions icon on the left sidebar and search for them):

- **Python** (by Microsoft)
 - **ES7+ React** (by dsznajder)
-

Phase 2 — Set up GitHub

GitHub is where your code lives online so both teammates can access it and you never lose your work.

1. Create a GitHub account at github.com if you don't have one.

2. Create a new repository

- Click the **+** button → New repository
- Name it **mahjongsensei**
- Set it to Private
- Tick "Add a README file"
- Click Create repository

3. Install Git on your computer at git-scm.com if it isn't already there. Check with:

```
bash
git --version
```

4. Clone the repo to your computer On your GitHub repo page, click the green Code button and copy the URL. Then in your terminal:

```
bash
git clone https://github.com/yourusername/mahjongsensei.git
cd mahjongsensei
```

Now you have a folder on your computer connected to GitHub.

Phase 3 — Set up the Django backend

Think of this as building the engine room of your app.

1. Open the project folder in VS Code

```
bash
code .
```

2. Create a virtual environment

A virtual environment is like a clean isolated box for your project's Python packages — so they don't conflict with anything else on your computer. Run this inside your project folder:

```
bash
python -m venv venv
```

Then activate it every time you work on the project:

```
bash
# Mac / Linux
source venv/bin/activate
```

```
# Windows
venv\Scripts\activate
```

You'll know it's active when you see (**venv**) at the start of your terminal line.

3. Install Django and the packages you need

```
bash
pip install django djangorestframework django-cors-headers psycopg2-binary
djangorestframework-simplejwt python-dotenv
```

This will take a minute. Once done, save a record of what you installed:

```
bash
pip freeze > requirements.txt
```

4. Create the Django project

```
bash
django-admin startproject backend
cd backend
```

Your folder structure now looks like:

```
mahjongsensei/
  venv/
  requirements.txt
  backend/
    manage.py
    backend/
      settings.py
      urls.py
```

5. Create your Django apps

In Django, an "app" is a module that handles one area of your project. Think of each one as a department:

bash

```
python manage.py startapp users
python manage.py startapp game
python manage.py startapp tutorial
```

- `users` — handles registration, login, profiles
- `game` — handles the Hand Helper, Trainer, Solo Play logic
- `tutorial` — handles the tutorial content and progress tracking

6. Tell Django about your apps

Open `backend/settings.py` in VS Code and find the `INSTALLED_APPS` list. Add your new apps:

python

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',      # add this
    'corsheaders',         # add this
    'users',               # add this
    'game',               # add this
    'tutorial',            # add this
]
```

7. Connect PostgreSQL

In the same `settings.py`, find the `DATABASES` section and replace it with:

python

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'mahjongsensei',
        'USER': 'postgres',
        'PASSWORD': 'your_password_here',
        'HOST': 'localhost',
        'PORT': '5432',
    }
```

```
}  
}
```

Replace `your_password_here` with the password you set when installing PostgreSQL.

You also need to create the database first. Open pgAdmin (it was installed with PostgreSQL), right-click Databases → Create → Database, and name it `mahjongsensei`.

8. Run your first migration

Migrations are how Django creates tables in your database based on your Python code. Run:

```
bash  
python manage.py migrate
```

This sets up all of Django's built-in tables (for users, sessions etc).

9. Check everything works

```
bash  
python manage.py runserver
```

Open `http://localhost:8000` in your browser. If you see the Django welcome page, your backend is running.

Phase 4 — Set up the React frontend

Open a **new terminal window** (keep Django running in the other one) and navigate back to your main project folder:

```
bash  
cd mahjongsensei  
npx create-react-app frontend  
cd frontend  
npm start
```

This will open `http://localhost:3000` in your browser with the default React page. You now have both running at the same time — React on port 3000, Django on port 8000.

Phase 5 — Connect React and Django

By default, React and Django can't talk to each other because browsers block requests between different addresses. You fix this with CORS.

In `backend/settings.py` add:

```
python
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # add at the very top
    ...
]

CORS_ALLOWED_ORIGINS = [
    'http://localhost:3000',
]
```

Now React can send requests to Django.

Phase 6 — Build the game engine first

Before building any API or UI, write your core Python logic. Create a new file at `backend/game/engine.py` and start building your classes:

```
python
class Tile:
    def __init__(self, suit, value):
        self.suit = suit # 'bamboo', 'circles', 'characters', 'honour'
        self.value = value # 1-9 for numbered suits, or 'east'/'west' etc

class Hand:
    def __init__(self, tiles):
        self.tiles = tiles # list of Tile objects

    def is_winning(self):
        # logic to check if hand is a winning hand
        pass

    def score(self):
        # logic to calculate points
        pass

class HandGenerator:
    def generate(self, difficulty='medium'):
        # generates a random valid hand
```

pass

```
class ValuationAlgorithm:
    def best_discard(self, hand):
        # evaluates each possible discard and returns the best one
        pass
```

Write these one at a time, test each one works before moving on.

The day-to-day workflow

Every time you sit down to work:

1. Open terminal, navigate to your project folder
2. Activate the virtual environment: `source venv/bin/activate`
3. Start Django: `cd backend && python3 manage.py runserver`
4. Open a second terminal, start React: `cd frontend && npm start`
5. Write code in VS Code
6. When done, commit to GitHub:

```
bash
git add .
git commit -m "describe what you did"
git push
```

What to build in what order

Don't try to build everything at once. Follow this sequence:

1. ☒ Get Django and React running and talking to each other
 2. ☒ Build the `Tile`, `Hand`, `HandGenerator`, `ValuationAlgorithm` classes and test them
 3. ☒ Build user registration and login
 4. ☒ Build the Tutorial feature (simplest feature, good for learning the pattern)
 5. ☒ Build the Hand Helper (uses `ValuationAlgorithm`)
 6. ☒ Build the Trainer (uses `HandGenerator` + `ValuationAlgorithm`)
 7. ☒ Build Solo Play (most complex, leave for last)
 8. ☒ Build Profile Dashboard, Leaderboard, and other extensions
-

The most important thing is not to get overwhelmed. Focus only on step 1 to start — just get Django and React running. Once that's done, everything else is just repeating the same pattern of: write some Python logic → expose it as a Django API endpoint → call it from React.

Summary of what each file in the apps do (can ignore first)

models.py — you will edit this

This is where you define your database tables as Python classes. For the **users** app you'll add your user profile model here. Think of each class as a table in PostgreSQL.

python

```
from django.db import models
```

```
from django.contrib.auth.models import User
```

```
class Profile(models.Model):
```

```
    user = models.OneToOneField(User, on_delete=models.CASCADE)
```

```
    username = models.CharField(max_length=50)
```

```
    created_at = models.DateTimeField(auto_now_add=True)
```

views.py — you will edit this

This is where you write what happens when a request hits an endpoint. For **users** this means your register and login logic.

admin.py — optional, useful for debugging

This lets you register your models so you can view and edit your database through Django's built-in admin panel at <http://localhost:8000/admin>. Useful during development to check if data is being saved correctly.

python

```
from django.contrib import admin
```

```
from .models import Profile
```

```
admin.site.register(Profile)
```

apps.py — leave it alone

Django uses this to configure the app internally. You already told Django about this app by adding it to `INSTALLED_APPS` in `settings.py` — nothing more needed here.

tests.py — you will edit this later

This is where you write your Pytest unit tests for the `users` app — testing that registration works, login works, wrong passwords are rejected etc. Leave it empty for now and come back to it once the features are built.

migrations/ folder — never edit this manually

Django automatically manages this folder. Every time you change your `models.py` and run `python manage.py makemigrations`, Django creates a new file in here describing the database change. Then `python manage.py migrate` applies those changes to PostgreSQL. You just run the commands — Django handles the files.

__init__.py — leave it alone

This is a Python convention that marks the folder as a Python package. It's always empty and you never touch it.

__pycache__/ — leave it alone

Python automatically creates this to store compiled versions of your code so it runs faster. Never edit it, and make sure it's in your `.gitignore`.

Summary of what you actually touch:

File	What you do
<code>models.py</code>	Write your database models
<code>views.py</code>	Write your API logic
<code>admin.py</code>	Register models for the admin panel
<code>tests.py</code>	Write tests later
Everything else	Leave alone